

SÉCURITÉ FIREBASE

Guide Complet du Développeur

Attaques, vulnérabilités et protection par Firebase Rules

Flutter + Firebase + Cloudinary

■ Ce guide décrit des techniques réelles d'attaque.

Son but est éducatif : comprendre pour mieux protéger.
Toute utilisation malveillante est illégale et contraire à l'éthique.

TABLE DES MATIÈRES

Chapitre 1	Introduction — Pourquoi l'UI ne protège rien	p. 3
Chapitre 2	Attaque 1 — Reverse Engineering (Décompilation APK)	p. 4
Chapitre 3	Attaque 2 — Interception des requêtes (MITM)	p. 5
Chapitre 4	Attaque 3 — Accès direct à l'API Firebase	p. 6
Chapitre 5	La vraie barrière — Les Firebase Security Rules	p. 7
Chapitre 6	Écrire des Rules solides — Exemples concrets	p. 9
Chapitre 7	Checklist de sécurité complète	p. 11
Chapitre 8	Conclusion & Ressources	p. 12

Lorsqu'un développeur construit une application mobile avec Flutter et Firebase, il est naturel de penser que les conditions écrites dans le code Dart suffisent à protéger les données. **C'est une erreur fondamentale.** Ce guide explique, de manière concrète et détaillée, pourquoi toute protection placée uniquement dans l'interface graphique est illusoire — et comment un attaquant peut la contourner en quelques minutes.

■ Principe de base en sécurité informatique

Tout code qui s'exécute sur le téléphone de l'utilisateur peut être lu, modifié et contourné par cet utilisateur. La sécurité doit toujours être imposée côté serveur, jamais côté client seul.

Scénario illustratif

Imaginons une application familiale Flutter/Firebase où seuls certains e-mails autorisés (stockés dans une collection Firestore *autorisations*) peuvent accéder aux vidéos privées. Le code Flutter vérifie :

```
if (doc.exists) { /* accorder l'accès */ } else { /* refuser */ }
```

À première vue, cela semble correct. En réalité, cette condition n'existe que sur l'écran du téléphone. Les trois chapitres suivants montrent comment un attaquant peut la rendre totalement inopérante.

2 Attaque 1 — Reverse Engineering

Le **reverse engineering** consiste à analyser une application compilée pour en comprendre le fonctionnement interne, extraire des secrets ou modifier son comportement. Cette technique est accessible à n'importe qui disposant d'un ordinateur et d'une connexion internet.

Comment fonctionne l'attaque ?

Étape 1	Récupération de l'APK	L'attaquant installe l'application depuis le Play Store ou obtient le fichier .apk directement. Sur Android, tout APK installé est accessible via ADB (Android Debug Bridge) avec la commande : <code>adb pull /data/app/.../.apk</code>
Étape 2	Décompilation avec JADX	JADX est un outil gratuit et open-source qui convertit le bytecode Android (Dalvik/DEX) en code Java/Kotlin lisible. Pour Flutter, des outils comme blutter ou reFlutter permettent de reconstruire le code Dart depuis le snapshot libapp.so contenu dans l'APK.
Étape 3	Extraction des secrets	Le fichier google-services.json et firebase_options.dart contiennent toutes les clés Firebase (apiKey, projectId, storageBucket, appId...). Ces clés sont visibles en clair dans le code décompilé.
Étape 4	Modification de la logique	L'attaquant supprime ou inverse la condition if (doc.exists), recompile l'APK avec apktool, le re-signe avec un certificat de test, et installe cette version modifiée sur son téléphone.

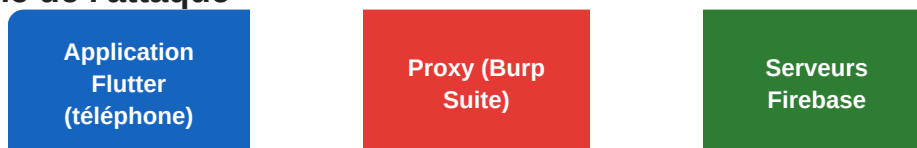
Outils utilisés par les attaquants

Outil	Plateforme	Usage	Niveau requis
JADX	Windows/Mac/Linux	Décompiler APK Android → Java/Kotlin	Débutant
apktool	Windows/Mac/Linux	Désassembler et recompiler APK	Intermédiaire
blutter	Linux	Reconstruire le code Dart depuis Flutter	Avancé
reFlutter	Windows/Mac/Linux	Analyser les snapshots Flutter	Avancé
Frida	Windows/Mac/Linux	Hooking dynamique à l'exécution	Expert

■ Les clés Firebase dans firebase_options.dart sont visibles dans tout APK décompilé. Ne placez jamais de secret

Une attaque **Man-in-the-Middle (MITM)** consiste à s'interposer entre l'application et le serveur pour lire, modifier ou rejouer les communications. Même si Firebase utilise HTTPS, cette attaque reste réalisable lorsque l'application ne valide pas correctement les certificats SSL.

Mécanisme de l'attaque



Flux de données : **App** → **proxy intercepte** → **Firebase**. L'attaquant voit et peut modifier chaque requête.

Ce que l'attaquant peut faire

- Lire toutes les réponses Firebase en clair (documents, URLs Cloudinary)
- Modifier la réponse du serveur : remplacer *false* par *true* dans un champ d'autorisation
- Rejouer des requêtes d'authentification (replay attack)

■ Extraire les tokens Firebase ID pour les réutiliser ailleurs

Logiciels utilisés

Logiciel	Prix	Capacités principales
Burp Suite (Community)	Gratuit	Proxy HTTP/HTTPS, modification des requêtes, replay
Charles Proxy	Payant (30j gratuit)	Proxy visuel, SSL proxying, throttling réseau
mitmproxy	Gratuit / Open-source	Proxy en ligne de commande, scriptable en Python
OWASP ZAP	Gratuit / Open-source	Scanner de vulnérabilités + proxy interactif

Contre-mesure côté Flutter

Activer le **Certificate Pinning** pour rejeter tout certificat non reconnu, même si l'appareil lui fait confiance :

```
// Avec le package http + SecurityContext
final context = SecurityContext(withTrustedRoots: false);
context.setTrustedCertificatesBytes(certBytes);
```

Attention : le certificate pinning seul n'est pas suffisant. Un attaquant expérimenté peut le contourner avec Frida. **La seule vraie protection reste les Firebase Rules côté serveur.**

Firebase est une **API REST publique**. Une fois en possession des clés Firebase (récupérables par reverse engineering), n'importe qui peut interagir directement avec votre base de données — sans jamais ouvrir votre application Flutter.

Script d'attaque Python (exemple pédagogique)

Voici ce qu'un attaquant peut écrire en moins de 20 lignes si les Firebase Rules sont absentes ou trop permissives :

```
import requests

# Clés extraites du fichier firebase_options.dart décompilé
API_KEY = "AIzaSy...extrait_du_APK..."
PROJECT_ID = "mon-projet-famille"

# 1. Connexion anonyme (si l'authentification anonyme est activée)
url_auth =
f"https://identitytoolkit.googleapis.com/v1/accounts:signIn?key={API_KEY}"
r = requests.post(url_auth, json={"returnSecureToken": True})
id_token = r.json()["idToken"]

# 2. Lecture directe de Firestore sans passer par Flutter
url_db = f"https://firestore.googleapis.com/v1/projects/{PROJECT_ID}/databases/(default)/documents/videos"
headers = {"Authorization": f"Bearer {id_token}"}
docs = requests.get(url_db, headers=headers).json()

# 3. Injection de faux documents (saturation Cloudinary)
for i in range(1000):
requests.post(url_db, headers=headers, json={"fields": {"url": {"stringValue":
f"faux_lien_{i}"}}})
```

Conséquences concrètes d'une absence de Rules

Impact	Description	Gravité
Vol de données	Lecture de toutes les vidéos et métadonnées privées	■ Critique
Saturation	Injection de milliers de documents → quota Firebase épuisé	■ Critique
Destruction	Suppression de tous les documents Firestore	■ Critique
Usurpation	Écriture de fausses données au nom d'autres utilisateurs	■ Élevé
Fuite emails	Enumération des e-mails autorisés dans la collection	■ Élevé

■ Sans Firebase Rules, les règles par défaut permettent souvent un accès total en lecture/écriture. Vérifiez imm

Les **Firebase Security Rules** sont des règles d'autorisation qui s'exécutent directement sur les serveurs de Google. Contrairement au code Flutter, elles ne peuvent **pas** être modifiées, contournées ou ignorées par un attaquant — peu importe ce qu'il fait sur son téléphone.

■ Pourquoi les Rules sont incontournables

- ✓ Elles s'exécutent sur les serveurs Google, pas sur le téléphone
- ✓ Un APK modifié ne peut pas les désactiver
- ✓ Un script Python externe est aussi bloqué par les Rules
- ✓ Même un token Firebase valide est inutile si les Rules refusent

Architecture des Rules Firestore

Les Rules Firestore utilisent une syntaxe déclarative. Chaque règle suit le schéma : **match /chemin/{variable} { allow opération: if condition; }**

Rule de base — Refuser tout par défaut

La première règle à écrire est toujours la plus restrictive possible :

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // RÈGLE PAR DÉFAUT : tout refuser
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

Comparaison : avec et sans Rules

Scenario	Sans Rules	Avec Rules
APK modifié	✗ Accès accordé	✓ Refusé côté serveur
Script Python externe	✗ Accès total	✓ Refusé côté serveur
Proxy MITM modifie réponse	✗ App trompée	✓ Requête invalide refusée
Token expiré/volé	✗ Accepté	✓ Vérifié par Firebase Auth
Injection de données	✗ Possible	✓ Bloquée par les Rules

Cas 1 — Autoriser uniquement les e-mails enregistrés

L'objectif : seuls les utilisateurs dont l'e-mail est présent dans la collection *autorisations* peuvent lire les vidéos. Cette vérification se fait **entièrement côté serveur**.

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    // Fonction helper – vérifie si l'email est autorisé
    function estAutorise() {
      return request.auth != null &&
        exists(/databases/{database}/documents/autorisations/{request.auth.token.email});
    }

    // Collection des vidéos – lecture seule pour les emails autorisés
    match /videos/{videoId} {
      allow read: if estAutorise();
      allow write: if false; // Aucune écriture depuis le client
    }

    // Collection des autorisations – lecture seule, écriture interdite
    match /autorisations/{email} {
      allow read: if request.auth != null &&
        request.auth.token.email == email;
      allow write: if false; // Géré uniquement par les Admin SDK
    }

    // Tout le reste : interdit
    match /{document=**} {
      allow read, write: if false;
    }
  }
}
```

Cas 2 — Règles Firebase Storage (Cloudinary alternatif)

Si vous utilisez Firebase Storage plutôt que Cloudinary, voici les règles correspondantes pour protéger les fichiers vidéo :

```
rules_version = '2';
service firebase.storage {
  match /b/{bucket}/o {
    // Vidéos – lecture uniquement pour les emails autorisés
    match /videos/{videoId} {
      allow read: if request.auth != null &&
        firestore.exists(/databases/(default)/documents/autorisations/
          {request.auth.token.email});
    }
  }
}
```



```

allow write: if false;
}
// Tout le reste : interdit
match /{allPaths=**} {
allow read, write: if false;
}
}
}

```

Cas 3 — Validation de la structure des données (écriture)

Les Rules peuvent aussi valider que les données écrites respectent un schéma précis, empêchant l'injection de champs malveillants :

```

match /commentaires/{commentId} {
allow create: if estAutorise()
&& request.resource.data.keys().hasOnly(['texte', 'auteur', 'date'])
&& request.resource.data.texte is string
&& request.resource.data.texte.size() <= 500
&& request.resource.data.auteur == request.auth.token.email;
}

```

■ Testez vos Rules avec le simulateur intégré dans la console Firebase avant de les déployer.

Utilisez cette checklist avant chaque mise en production. Chaque point non coché représente une vulnérabilité potentielle.

■ Firebase Rules

- Firestore Rules déployées et testées
- Règle par défaut : allow read, write: if false
- Accès restreint par e-mail ou UID Firebase
- Écriture cliente désactivée pour les collections critiques
- Validation de la structure des données en écriture
- Firebase Storage Rules configurées (si utilisé)

■ Authentification

- Firebase Authentication activé (Email/Password ou autre)
- Authentification anonyme désactivée si non nécessaire
- Vérification de request.auth != null dans toutes les Rules
- Token ID vérifié côté serveur pour les opérations Admin

■ Code Flutter / Client

- Aucune logique de sécurité critique dans le code Flutter seul
- Clés API Firebase non commises dans un dépôt public (GitHub)
- Certificate pinning activé (optionnel mais recommandé)
- Obfuscation du code Flutter activée en release

■ Infrastructure

- Quota Firebase configuré (éviter la saturation)
- Alertes de facturation activées dans Google Cloud
- Logs d'audit Firebase activés et surveillés
- Admin SDK utilisé côté serveur pour les opérations privilégiées

■ Les 3 principes à retenir absolument

1. **L'UI ne protège rien.** Toute condition dans le code Flutter peut être supprimée ou contournée en moins d'une heure par un attaquant déterminé.
2. **Les Firebase Rules sont la seule vraie barrière.** Elles s'exécutent côté serveur, sont incontournables depuis le client, et couvrent tous les vecteurs d'attaque décrits dans ce guide.
3. **Par défaut, refuser tout.** Commencez par *allow read, write: if false* et ouvrez progressivement uniquement ce qui est nécessaire.

Ressources officielles

Documentation	URL
Firebase Security Rules — Guide officiel	firebase.google.com/docs/rules
Firebase Security Rules — Référence Firestore	firebase.google.com/docs/firestore/security/get-started
Firebase Security Rules — Simulateur	console.firebase.google.com → Firestore → Rules → Simulator
OWASP Mobile Security Testing Guide	owasp.org/www-project-mobile-security-testing-guide
Flutter Security Best Practices	docs.flutter.dev/deployment/obfuscate

*Ce guide a été rédigé dans un but **strictement éducatif**. La compréhension des techniques d'attaque est indispensable pour construire des défenses efficaces. Toute utilisation de ces techniques contre des systèmes sans autorisation explicite est illégale.*